BANK PROJECT

*/*I. Initial Exploratory Analysis*

*Objective: Understand the dataset's structure and key characteristics to establish a foundation for deeper analysis.**

/*I.A. Data types for columns in the 'customers' table

● Hint: Use the information_schema.columns view to query the data types by filtering on the 'customers' table and 'bank' schema.*/

```
show columns from ccustomers;

select column_name,data_type

from information_schema.columns

where table_schema = 'bank'and table_name = 'customers';
```

Result Grid | Filter Rows: | Export:

Field	Type	Null	Key	Default	Extra
customer_id	int	YES		NULL	
first_name	text	YES		NULL	
last_name	text	YES		NULL	
email	text	YES		NULL	
phone	text	YES		NULL	
address	text	YES		NULL	
date_of_birth	text	YES		NULL	
ssn	text	YES		NULL	
created_at	text	YES		NULL	

-- I.B. Identify the date range covered in the bank customer and transaction data by finding:

-- ● Hint: Use MIN and MAX with STR_TO_DATE on the created_at column from the customers table and transaction_date from the transactions table, joined via accounts.

```
select min(str_to_date(c.created_at, '%Y-%m-%d')) as customer_start_date,

max(str_to_date(c.created_at, '%Y-%m-%d')) as customer_end_date,

min(str_to_date(t.transaction_date, '%Y-%m-%d')) as transaction_start_date,

max(str_to_date(t.transaction_date, '%Y-%m-%d')) as transaction_end_date

from ccustomers as  c

join accounts as a
```

```
on c.customer_id = a.customer_id
join transactions as  t
on a.account_id =
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_start_date	customer_end_date	transaction_start_date	transaction_end_date
2001-02-20	2031-03-20	2001-01-20	2031-10-20

```
t.account_id;
-- I.C. List the name of each branch, the type of account, and the status of the account,
-- along with the total number of accounts for each combination, and show the results
-- sorted by branch name and then by the number of accounts from highest to lowest.
-- ● Hint: Use a JOIN between the accounts and branches tables, group by
-- branch_name, account_type, and status, and count account_id.*/
select b.branch_name,a.account_type,a.status,count(a.account_id) as total_accounts
from accounts a
join branches b
on a.branch_id = b.branch_id
group by b.branch_name,a.account_type,a.status
order by b.branch_name,total_accounts
```

Result Grid | Filter Rows: | Export:

branch_name	account_type	status	total_accounts
Aventura Plaza	FIXED_DEPOSIT	FROZEN	4
Aventura Plaza	SAVINGS	ACTIVE	3
Aventura Plaza	SAVINGS	FROZEN	3
Aventura Plaza	CHECKING	CLOSED	2
Aventura Plaza	SAVINGS	CLOSED	2
Aventura Plaza	CHECKING	ACTIVE	2
Aventura Plaza	FIXED_DEPOSIT	ACTIVE	1
Aventura Plaza	FIXED_DEPOSIT	CLOSED	1
Brickell Avenue	SAVINGS	ACTIVE	4
Brickell Avenue	CHECKING	CLOSED	4
Brickell Avenue	SAVINGS	CLOSED	3
Brickell Avenue	CHECKING	ACTIVE	3

Result 61

```
desc;
```

-- II. Customer Behavior Analysis

-- Objective: Understand customer engagement and transaction patterns to

-- improve retention and product offerings.

-- II.A. Are high-balance accounts associated with higher transaction frequency? Find

-- the average number of transactions for each customer balance category (High

-- Balance >50K, Medium Balance 10K-50K, Low Balance <=10K), along with the number

-- of customers and average balance in each category, sorted by average transaction

-- count from highest to lowest.

-- ● Hint: Use a CTE or subquery to calculate average balance and transaction

-- count per customer, then categorise balances with CASE and aggregate by

-- category with AVG on transaction count, ordering by average frequency.

```
with cte as (select customer_id,(count(transaction_id)) as count_transaction,balance,

case when  balance>50000 then 'High balance'

when  balance between 10000 and 50000 then 'Medium balance' else 'Low Balance' end as
balance_category

from accounts as a

left join transactions as t

on a.account_id=t.account_id

group by customer_id,balance)

select balance_category,count(customer_id),avg(balance),avg(count_transaction) from cte

group by balance_category

order by balance_category
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

balance_category	count(customer_id)	avg(balance)	avg(count_transaction)
Medium balance	73	32443.73082191781	0.9589
Low Balance	19	4882.460526315789	1.1579
High balance	108	77564.66796296298	1.0000

```
desc;
```

-- II.B. Find the top two months with the highest transaction counts for each account

-- type in 2025, showing the account type, month name, transaction count, and total

-- transaction amount, sorted by account type and rank. accoutn type ----------top two month+ highest transcation count+ amount

-- ● Hint: Use a CTE to group transactions by account type and month name/year,

-- then apply a window function over partition by account type ordered by

-- transaction count DESC to select top 2 ranks.

```
with cte as (select a.account_type as aaccount_type,month(str_to_date(transaction_date,'%d-%m-%Y') )as months,

year(str_to_date(transaction_date,'%d-%m-%Y') )as years,count(t.transaction_id) as counts ,

 a.account_type,sum(t.amount) as transactionn_amount

from accounts as a

left join transactions as t

on a.account_id=t.account_id

group by a.account_type,months,years),


cte2 as (select aaccount_type,counts,transactionn_amount,months,years,rank() over (partition by aaccount_type order by counts desc)as ranks

 from cte

 where years = 2025

 )

 select aaccount_type,months,counts,ranks,transactionn_amount

 from cte2

 where ranks<=2

 order by aaccount_type, ranks;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

aaccount_type	months	counts	ranks	transactionn_amount
CHECKING	3	3	1	13687.25
CHECKING	12	1	2	6697.89
CHECKING	7	1	2	7551.88
CHECKING	1	1	2	9662.33
CHECKING	4	1	2	8973.11
CHECKING	2	1	2	9391.14
FIXED_DEPOSIT	4	3	1	18265.26
FIXED_DEPOSIT	12	2	2	5852.73
FIXED_DEPOSIT	8	2	2	4581.79
FIXED_DEPOSIT	9	2	2	9514.91
SAVINGS	6	3	1	14196.39
SAVINGS	12	2	2	7019.17

Result 63

-- II.C. Count the total number of credit card transactions for each age group (Under 30,

-- 30-50, Over 50) as of June 1, 2025, and list them in order from highest to lowest

-- transaction count.

-- ● Hint: Use DATEDIFF with a fixed date ('2025-06-01') and date_of_birth to

-- calculate age, categorise with CASE into age groups, join customers to

-- accounts, cards, and transactions filtering on CREDIT card type, then count

-- transactions per group.

```
select count(t.transaction_id) as transaction_count,

case when timestampdiff(year, c.date_of_birth, '2025-06-01') < 30 then 'under 30'

when timestampdiff(year, c.date_of_birth, '2025-06-01') between 30 and 50 then '30-50'else 'over 50'end as age_category

from transactions as t

left join accounts as a

on t.account_id=a.account_id

left join ccustomers as c

on a.customer_id=c.customer_id

inner join cards as cd

on a.account_id=cd.account_id

where cd.card_type='CREDIT'

group by age_category

order by transaction_count desc;
```



Result Grid | Filter Rows:

transaction_count	age_category
123	over 50

-- II.D. List the top 10 customers with the highest total balances across their active

-- accounts, showing their name, email, total balance, number of accounts, and earliest

-- account opening date, sorted by total balance from highest to lowest.

-- ● Hint: Use a CTE or subquery to join customers and accounts filtering on

-- ACTIVE status, aggregate and then order by total balance DESC and limit to 10.

```
with cte as (select c.customer_id,concat(c.first_name,' ',c.last_name) as
full_name,c.email,count(a.account_id) as acc_count,sum(a.balance) as
total_amt,min(a.opened_date) as opendate

from accounts as a
```

```
inner join ccustomers as c
on a.customer_id=c.customer_id
where status='active'
group by a.customer_id,full_name,c.email
order by sum(a.balance) desc limit 10)
select * from cte ;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	full_name	email	acc_count	total_amt	opendate
41	David Patel	smithjulieexample.org	3	199608.72999999998	10-06-2021
154	Diana Hebert	brandon39example.net	2	154251.97	20-12-2021
94	Beverly Wagner	phyllismeyerexample.net	2	140071.94	17-10-2024
151	Joan Johnson	paulproctorexample.org	2	140069.64	24-08-2023
90	Justin Marquez	smithjasonexample.org	2	121257.86	08-06-2022
105	Loretta Barnes	lindsey69example.org	2	117072.95	22-02-2021
12	Rebecca Gardner	rodriguezsierraexample.net	1	98393.6	21-12-2023
200	Katelyn Davis	thomaskevinexample.net	1	96472.08	NULL
30	Charles Harrington	jsmithexample.com	1	96289.33	NULL
11	Eric Campbell	mcmillandennisexample.org	1	94961.17	NULL

Result 67

```
-- II.E. List customers with more than one active loan, showing their customer ID, name,
-- number of active loans, and total loan amount, sorted by total loan amount from
-- highest to lowest.
-- ● Hint: Join loans and customers filtering on ACTIVE loan status, group by
-- customer_id and name, use aggregate functions, then filter > 1, ordered by
-- total amount DESC.

select * from loans;
select c.customer_id,l.status,concat(c.first_name,' ',c.last_name) as full_name,count(l.loan_id) as loan_count,sum(l.loan_amount) as amt_loan
from ccustomers as c
inner join loans as l
on c.customer_id=l.customer_id
where l.status='active'
group by c.customer_id,full_name
```

having count(l.loan_id)>1

order by sum(l.loan_amount) desc;



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	status	full_name	loan_count	amt_loan
26	ACTIVE	Anthony Perez	2	752963.88
57	ACTIVE	Shannon Pierce	2	719301.4099999999
105	ACTIVE	Loretta Barnes	2	533704.88
127	ACTIVE	Laura Grant	2	454377.42
183	ACTIVE	Monica Smith	2	367885.54000000004

-- III. Branch Performance Analysis

-- Objective: Evaluate branch efficiency and contribution to business outcomes.

-- III.A. List the top two branches with the highest number of accounts opened each

-- year, showing the branch name, year, and number of accounts, sorted by year and

-- then by number of accounts from highest to lowest.

-- ● Hint: Use a CTE to group accounts by branch and year (using YEAR on

-- opened_date), then apply appropriate window function over partition by year

-- ordered by count DESC to select the top 2 ranks per year.

```
with cte as (select b.branch_id ,b.branch_name,count(a.opened_date) as
how_many_acc_opened,year(str_to_date(a.opened_date,'%Y-%m-%d')) as years

from branches as b

left join accounts as a

on a.branch_id=b.branch_id

group by branch_id,branch_name,years ),

cte1 as (

select branch_id,how_many_acc_opened,branch_name,years,rank() over (partition by years order
by how_many_acc_opened desc)as ranks from cte)

 select * from cte1

 where ranks<=2;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

branch_id	how_many_acc_opened	branch_name	years	ranks
9	2	West Palm	2001	1
1	1	Downtown Miami	2001	2
2	1	South Beach	2001	2
5	1	Dade Central	2001	2
6	1	Brickell Avenue	2001	2
8	1	Aventura Plaza	2001	2
4	1	Kendall Branch	2002	1
9	1	West Palm	2002	1
6	1	Brickell Avenue	2004	1
8	1	Aventura Plaza	2004	1
9	1	West Palm	2004	1
1	2	Downtown Miami	2005	1

Result 69

-- III.B. Show the name of each branch along with the number of employees, number

-- of accounts, total transaction amount, and a performance rank based on transaction

-- amount, sorted by performance rank from highest to lowest.

-- ● Hint: Use JOINs from branches to employees, accounts, and transactions,

-- aggregate for employees/accounts and transactions, then, with window

-- functions, order the total amount DESC.

```
with cte as (select b.branch_id,b.branch_name,count(distinct e.employee_id) ,count(distinct
a.account_id),sum(t.amount) as total

from accounts as a

left join transactions as t

on a.account_id=t.account_id

left join eemployees as e

on a.branch_id=e.branch_id

left join branches as b

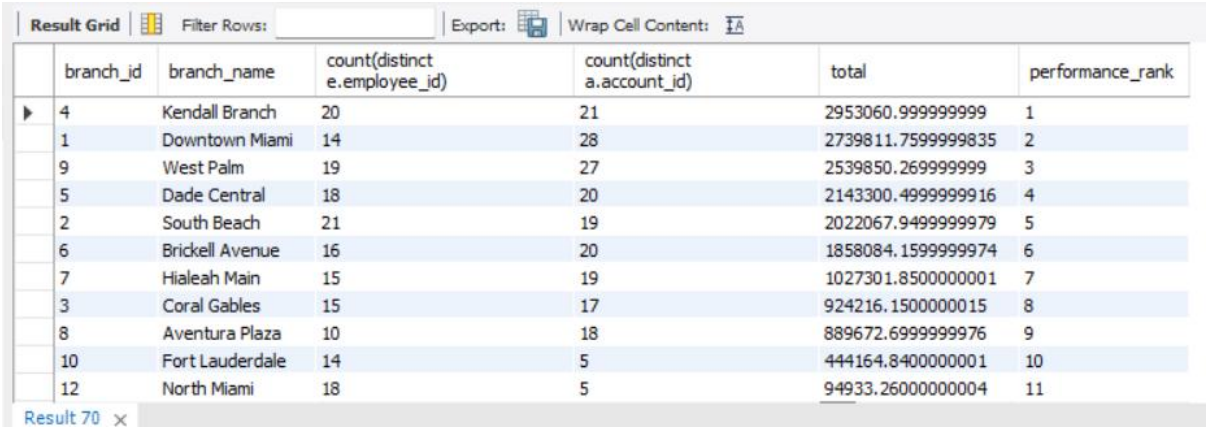
on a.branch_id=b.branch_id

group by b.branch_id,b.branch_name

order by sum(t.amount) desc),

cte1 as( select * , rank() over (order by total desc) as performance_rank from cte )

select * from cte1;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

branch_id	branch_name	count(distinct e.employee_id)	count(distinct a.account_id)	total	performance_rank
4	Kendall Branch	20	21	2953060.999999999	1
1	Downtown Miami	14	28	2739811.7599999835	2
9	West Palm	19	27	2539850.269999999	3
5	Dade Central	18	20	2143300.4999999916	4
2	South Beach	21	19	2022067.9499999979	5
6	Brickell Avenue	16	20	1858084.1599999974	6
7	Hialeah Main	15	19	1027301.8500000001	7
3	Coral Gables	15	17	924216.1500000015	8
8	Aventura Plaza	10	18	889672.6999999976	9
10	Fort Lauderdale	14	5	444164.8400000001	10
12	North Miami	18	5	94933.26000000004	11

Result 70

-- III.C. List each branch's name, year, total transaction amount, cumulative transaction

-- amount over the years, and the average transaction amount per year across all

-- branches, sorted by branch name and year.

-- ● Hint: Use a CTE to group transactions by branch and year (using YEAR on

-- transaction_date), then apply SUM with window over partition by branch

-- ordered by year for cumulative and AVG over partition by year for average.

```
with cte as

(select b.branch_name,year(str_to_date(t.transaction_date, '%Y-%m-%d')) as years,sum(t.amount) as total_trans_amt

from branches as b

left join accounts as a

on b.branch_id = a.branch_id

left join transactions as t

on a.account_id = t.account_id

group by b.branch_name, years

)

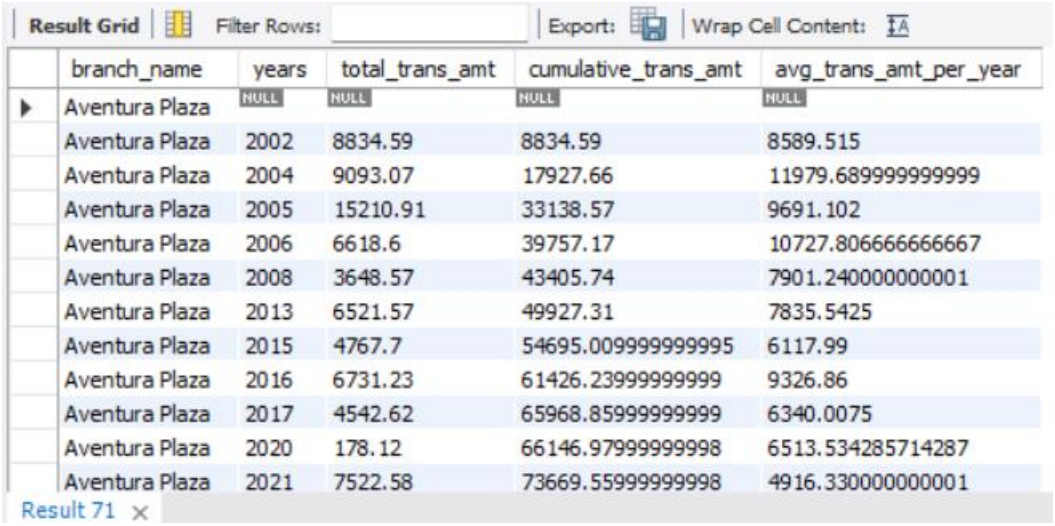
select branch_name,years,total_trans_amt,

sum(total_trans_amt) over (partition by branch_name order by years rows between unbounded preceding and current row) as cumulative_trans_amt,

avg(total_trans_amt) over (partition by years) as avg_trans_amt_per_year

from cte

order by branch_name, years;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

branch_name	years	total_trans_amt	cumulative_trans_amt	avg_trans_amt_per_year
Aventura Plaza	NULL	NULL	NULL	NULL
Aventura Plaza	2002	8834.59	8834.59	8589.515
Aventura Plaza	2004	9093.07	17927.66	11979.689999999999
Aventura Plaza	2005	15210.91	33138.57	9691.102
Aventura Plaza	2006	6618.6	39757.17	10727.806666666667
Aventura Plaza	2008	3648.57	43405.74	7901.240000000001
Aventura Plaza	2013	6521.57	49927.31	7835.5425
Aventura Plaza	2015	4767.7	54695.009999999995	6117.99
Aventura Plaza	2016	6731.23	61426.23999999999	9326.86
Aventura Plaza	2017	4542.62	65968.85999999999	6340.0075
Aventura Plaza	2020	178.12	66146.97999999998	6513.534285714287
Aventura Plaza	2021	7522.58	73669.55999999998	4916.330000000001

Result 71

-- III.D. List each branch's ID and name along with the total loan amount issued,

-- number of active loans, average interest rate, and percentage of loans paid, sorted by

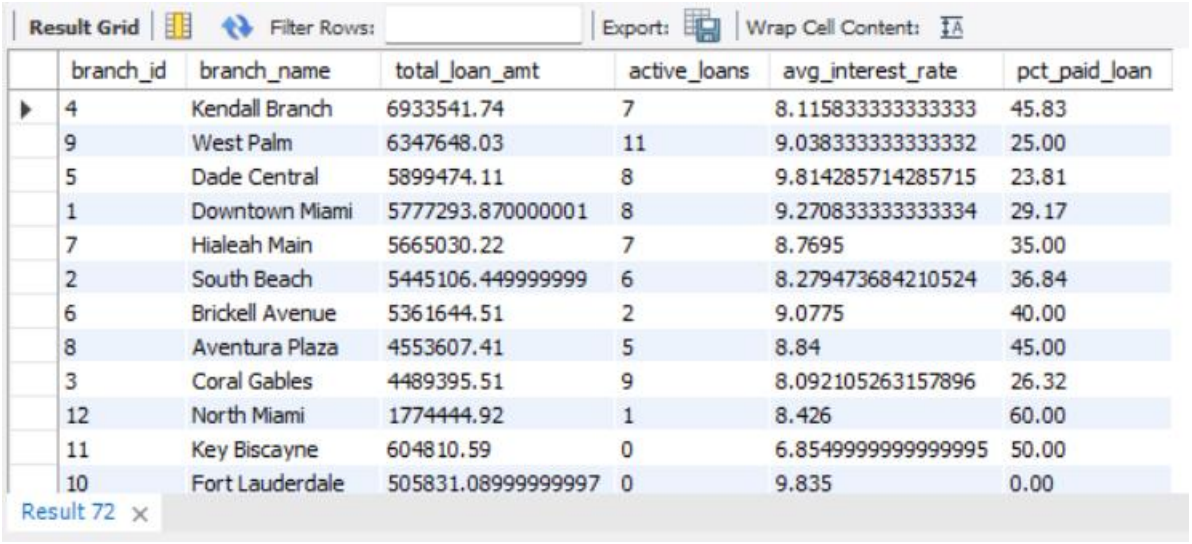
-- total loan amount from highest to lowest.

-- ● Hint: JOIN loans to branches on branch_id, aggregate loan_amount, SUM for

-- ACTIVE status count, AVG(interest_rate), and ROUND(100 * SUM for PAID /

-- COUNT(*)) for percentage, ordered by total issued DESC.

```
select b.branch_id,b.branch_name,sum(l.loan_amount) as total_loan_amt,
sum(case when l.status = 'active' then 1 else 0 end) as active_loans,
avg(l.interest_rate) as avg_interest_rate,
round(100.0 * sum(case when l.status = 'paid' then 1 else 0 end)/ count(l.loan_id),2) as
pct_paid_loan
from branches as b
inner join loans as l
on b.branch_id = l.branch_id
group by b.branch_id, b.branch_name
order by total_loan_amt desc;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

branch_id	branch_name	total_loan_amt	active_loans	avg_interest_rate	pct_paid_loan
4	Kendall Branch	6933541.74	7	8.115833333333333	45.83
9	West Palm	6347648.03	11	9.038333333333332	25.00
5	Dade Central	5899474.11	8	9.814285714285715	23.81
1	Downtown Miami	5777293.870000001	8	9.270833333333334	29.17
7	Hialeah Main	5665030.22	7	8.7695	35.00
2	South Beach	5445106.449999999	6	8.279473684210524	36.84
6	Brickell Avenue	5361644.51	2	9.0775	40.00
8	Aventura Plaza	4553607.41	5	8.84	45.00
3	Coral Gables	4489395.51	9	8.092105263157896	26.32
12	North Miami	1774444.92	1	8.426	60.00
11	Key Biscayne	604810.59	0	6.8549999999999995	50.00
10	Fort Lauderdale	505831.08999999997	0	9.835	0.00

Result 72

-- IV. Financial Risk and Revenue Analysis

-- Objective: Assess financial performance and risk exposure through loans and

-- transactions.

-- IV.A. List the top three branches with the highest loan default rates for each loan

-- type, showing the loan type, branch name, total loans issued, number of defaults,

-- and default rate percentage, sorted by loan type and default rate from highest to

-- lowest.

-- ● Hint: Use a CTE to aggregate total loans and defaulted loans per loan_type and

-- branch_name, then apply a window function over loan_type ordered by

-- default rate DESC to select the top 3 ranks.

```
with cte as (select b.branch_id,b.branch_name as branch_names,loan_type,count(l.loan_id) as
loan_issued,sum(case when l.status='defaulted' then 1 else 0 end) as defaults_rate

from branches as b

inner join loans as l

on b.branch_id = l.branch_id

group by branch_id,branch_name,loan_type),

 cte1 as(select loan_type,branch_names,loan_issued,defaults_rate,rank() over (partition by
loan_type order by (defaults_rate/loan_issued) *100 desc) as rn ,
```

(defaults_rate/loan_issued) *100 as pct_of_defualt from cte)

select * from cte1

where rn<=3

order by loan_type,pct_of_defualt desc ;



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

loan_type	branch_names	loan_issued	defaults_rate	rn	pct_of_defualt
AUTO	Brickell Avenue	5	3	1	60.0000
AUTO	Downtown Miami	6	3	2	50.0000
AUTO	North Miami	2	1	2	50.0000
AUTO	Aventura Plaza	4	2	2	50.0000
AUTO	Dade Central	2	1	2	50.0000
BUSINESS	Fort Lauderdale	2	2	1	100.0000
BUSINESS	Hialeah Main	6	4	2	66.6700
BUSINESS	West Palm	8	5	3	62.5000
MORTGAGE	Key Biscayne	1	1	1	100.0000
MORTGAGE	Coral Gables	3	2	2	66.6700
MORTGAGE	Kendall Branch	8	5	3	62.5000
PERSONAL	South Beach	4	4	1	100.0000

Result 73

-- IV.B. List each branch's name, the total interest revenue from active loans, and the

-- percentage contribution to the total interest revenue across all branches, sorted by

-- interest revenue from highest to lowest.

-- ● Contribution to Total Revenue (%)= (Branch Interest Revenue / Total Interest

-- Revenue of All Branches)×100

-- ● Hint: Use a CTE to calculate interest revenue as SUM(loan_amount *

-- interest_rate / 100) for ACTIVE loans per branch, then compute the percentage

-- using window SUM over all revenue.

with cte as(select b.branch_name,sum(l.loan_amount * l.interest_rate / 100.0)as total_interest_revenue

from branches as b

inner join loans as l

on b.branch_id = l.branch_id

where l.status='active'

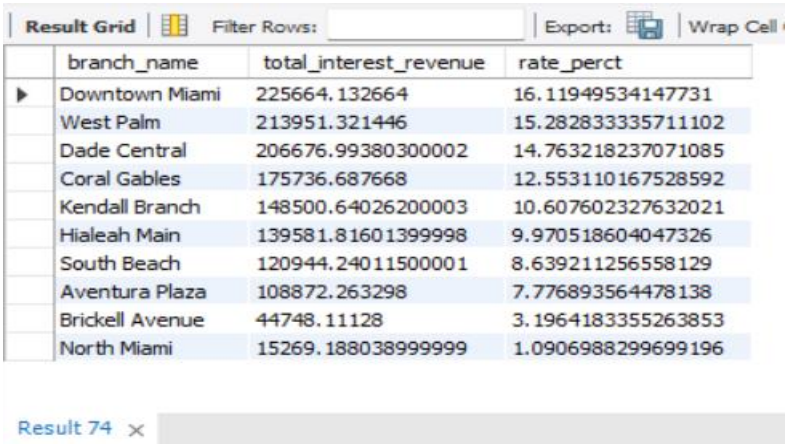
group by b.branch_name

order by total_interest_revenue desc)

```
select
branch_name,total_interest_revenue,total_interest_revenue/sum(total_interest_revenue)over () *
100  as rate_perct

from cte

order by total_interest_revenue desc;
```



Result Grid | Filter Rows: | Export: | Wrap Cell

branch_name	total_interest_revenue	rate_perct
Downtown Miami	225664.132664	16.11949534147731
West Palm	213951.321446	15.282833335711102
Dade Central	206676.99380300002	14.763218237071085
Coral Gables	175736.687668	12.553110167528592
Kendall Branch	148500.64026200003	10.607602327632021
Hialeah Main	139581.81601399998	9.970518604047326
South Beach	120944.24011500001	8.639211256558129
Aventura Plaza	108872.263298	7.776893564478138
Brickell Avenue	44748.11128	3.1964183355263853
North Miami	15269.188038999999	1.0906988299699196

Result 74 ×

-- IV.C. List the loan ID, customer name, loan amount, and number of days since the

-- last payment for active loans with no payment or payments overdue by more than

-- 180 days as of April 30, 2025, sorted by days overdue from highest to lowest.

-- ● Hint: Use a CTE to find the maximum payment_date per ACTIVE loan joined to

-- loan_payments, then filter where last_payment_date based on the conditions,

-- joining to customers for name.

```
with cte as (select l.loan_id,customer_id,l.loan_amount,  max(STR_TO_DATE(lp.payment_date, '%d-
%m-%Y ))  as last_payment_date

from loans as l

left join loan_payments as lp

on l.loan_id=lp.loan_id

where l.status='active'

group by l.loan_id,customer_id,l.loan_amount)

select y.loan_id,y.loan_amount,concat(c.first_name,' ',c.last_name) as full_name,DATEDIFF('2025-04-
30', y.last_payment_date) as days_overdue

from cte as y

inner join ccustomers as c

on y.customer_id=c.customer_id
```

where y.last_payment_date is null

or y.last_payment_date<DATE_SUB('2025-04-30', interval 180 day)

order by days_overdue desc;



Result Grid | Filter Rows: | Export:

loan_id	loan_amount	full_name	days_overdue
66	232972.01	Brandy Walker	1795
130	70748.73	Loretta Barnes	1702
160	111038.7	Sean Myers	1665
48	446620.39	Scott Smith	1466
117	228961.07	Audrey Tucker	1445
139	8874.76	Brian Gutierrez	1436
73	136187.3	Sandy Wilson	1362
8	326420.3	Anthony Perez	1273
59	383682.15	Tyrone Simon	1209
184	96810.61	Robert Cooper	1206
42	122748.24	Laura Grant	1201
29	408476.72	Shannon Pierce	1117

Result 75

select * from loan_payments;

-- IV.D. List the year, total transaction amount, previous year's total amount, and the

-- percentage increase in transaction amounts from January to August for each year,

-- sorted by year, excluding the first year's increase.

-- ● Hint: Use a CTE to calculate the total amount per year for transactions where

-- the month is BETWEEN 1 AND 8, then apply a window function to calculate

-- (current - previous) / previous * 100 percentage increase

with cte as (select year(str_to_date(transaction_date, '%d-%m-%Y %H:%i:%s')) as years,

sum(amount) as total_transaction_amount

from transactions

where month(str_to_date(transaction_date, '%d-%m-%Y %H:%i:%s')) between 1 and 8

group by year(str_to_date(transaction_date, '%d-%m-%Y %H:%i:%s'))

),

cte2 as

(select years,total_transaction_amount,lag(total_transaction_amount) over (order by years) as previous_year_amount

from cte)

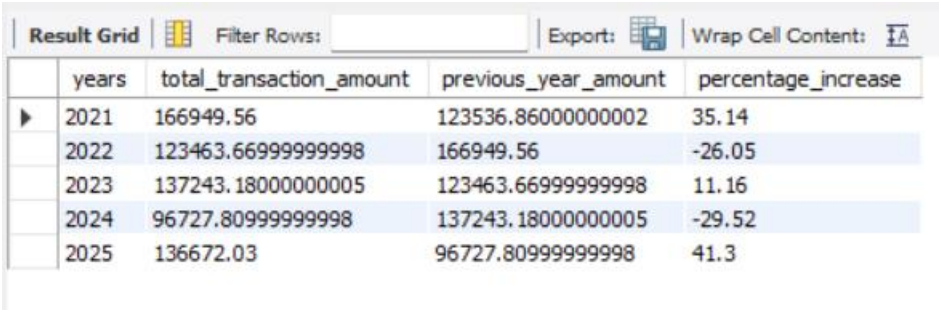
select years,total_transaction_amount,previous_year_amount,

round((total_transaction_amount - previous_year_amount)/ previous_year_amount * 100,2) as percentage_increase

from cte2

where previous_year_amount is not null

order by years;



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

years	total_transaction_amount	previous_year_amount	percentage_increase
2021	166949.56	123536.86000000002	35.14
2022	123463.66999999998	166949.56	-26.05
2023	137243.18000000005	123463.66999999998	11.16
2024	96727.80999999998	137243.18000000005	-29.52
2025	136672.03	96727.80999999998	41.3

-- V. Customer Service and Interaction Analysis

-- Objective: Optimize customer service processes and improve satisfaction.

-- V.A. List each branch name, total number of interactions, number of complaints,

-- complaint rate percentage, number of employees, and their positions, sorted by

-- complaint rate from highest to lowest.

-- ● Hint: Join customer_interactions, employees, and branches tables, group by

-- branch_name, count total interactions and complaints with CASE, calculate

-- complaint rate, and list distinct employee positions with GROUP_CONCAT.

select b.branch_name,count(ci.interaction_id),sum(case when ci.interaction_type='COMPLAINT' then 1 else 0 end) as no_of_complaint,

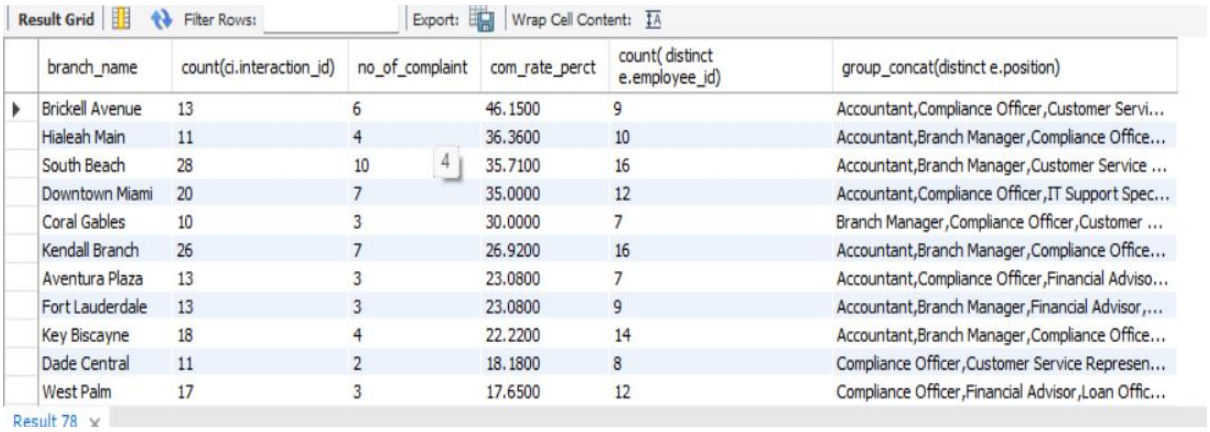
sum(case when ci.interaction_type='COMPLAINT' then 1 else 0 end) /count(ci.interaction_type)* 100 as com_rate_perct,count(distinct e.employee_id),

group_concat(distinct e.position)

from customer_interactions as ci

left join eemployees as e

```
on ci.employee_id=e.employee_id
inner join branches as b
on e.branch_id=b.branch_id
group by b.branch_name
order by com_rate_perct desc ;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

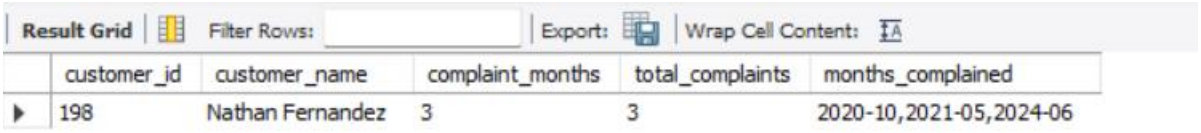
branch_name	count(ci.interaction_id)	no_of_complaint	com_rate_perct	count(distinct e.employee_id)	group_concat(distinct e.position)
Brickell Avenue	13	6	46.1500	9	Accountant,Compliance Officer,Customer Servi...
Hialeah Main	11	4	36.3600	10	Accountant,Branch Manager,Compliance Office...
South Beach	28	10	35.7100	16	Accountant,Branch Manager,Customer Service ...
Downtown Miami	20	7	35.0000	12	Accountant,Compliance Officer,IT Support Spec...
Coral Gables	10	3	30.0000	7	Branch Manager,Compliance Officer,Customer ...
Kendall Branch	26	7	26.9200	16	Accountant,Branch Manager,Compliance Office...
Aventura Plaza	13	3	23.0800	7	Accountant,Compliance Officer,Financial Adviso...
Fort Lauderdale	13	3	23.0800	9	Accountant,Branch Manager,Financial Advisor,...
Key Biscayne	18	4	22.2200	14	Accountant,Branch Manager,Compliance Office...
Dade Central	11	2	18.1800	8	Compliance Officer,Customer Service Represen...
West Palm	17	3	17.6500	12	Compliance Officer,Financial Advisor,Loan Offic...

Result 78

```
select * from customer_interactions;


-- V.B. List customers who made at least one complaint in three or more months,
-- showing their ID, name, number of complaint months, total complaints, and the
-- months they complained, sorted by number of months and total complaints from
-- highest to lowest.
-- ● Hint: Join customer_interactions and customers filtering on 'COMPLAINT' type,
-- group by customer_id and name, count distinct months with DATE_FORMAT,
-- and filter for >= 3 months.
with cte as (
select c.customer_id,c.first_name,c.last_name,ci.interaction_id,
str_to_date(ci.interaction_date, '%d-%m-%Y ') as converted_date
from customer_interactions as ci
inner join ccustomers as c
on ci.customer_id = c.customer_id
where ci.interaction_type = 'COMPLAINT'
)
select customer_id,
```

```
concat(first_name, ' ', last_name) as customer_name,
count(distinct date_format(converted_date, '%Y-%m')) as complaint_months,
count(interaction_id) as total_complaints,
group_concat(distinct date_format(converted_date, '%Y-%m')
order by date_format(converted_date, '%Y-%m')
) as months_complained
from cte
group by customer_id, first_name, last_name
having count(distinct date_format(converted_date, '%Y-%m')) >= 3
order by complaint_months desc, total_complaints desc;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	complaint_months	total_complaints	months_complained
198	Nathan Fernandez	3	3	2020-10,2021-05,2024-06

```
-- VI. Card Portfolio Optimisation Analysis
-- Objective: Optimize card offerings and usage to enhance customer value and
-- financial performance.

-- VI.A. List customers with an active credit card where the account balance is 90% or
-- more of the credit limit, showing their ID, name, card number, credit limit, balance,
-- and percentage of credit used.
-- ● Hint: Join customers, accounts, and cards tables, filter for card_type and
-- balance, then calculate percentage.
select
c.customer_id,c.first_name,c.last_name,cr.card_number,cr.credit_limit,a.balance,round(a.balance/cr.
credit_limit * 100,2) as perct_cr_card_used
from ccustomers as c
inner join accounts as a
on c.customer_id=a.customer_id
inner join cards as cr
on a.account_id=cr.account_id
```

where card_status='active' and card_type='credit'and (a.balance >= (cr.credit_limit*0.9))

order by perct_cr_card_used desc;



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	first_name	last_name	card_number	credit_limit	balance	perct_cr_card_used
83	Gary	Cunningham	3570866882634353	2303.75	98077.46	4257.3
62	Kenneth	Morrison	4105123828258673	1327.62	43987.26	3313.24
146	Abigail	Cole	570541152627	4867.61	43915.34	902.2
164	Amanda	Ball	30464887719062	8810.94	69031.35	783.47
83	Gary	Cunningham	30437489691087	13610.09	98077.46	720.62
106	Mary	Scott	4215107029662	12470.56	86383.67	692.7
83	Gary	Cunningham	4752500743292717	14161.39	98077.46	692.57
179	Amy	Bates	4458685014294019	8588.61	55441.25	645.52
151	Joan	Johnson	30223320616321	10167.49	59679.07	586.96
81	Shannon	Jordan	3596705466889373	10645.59	60620.89	569.45
182	Cassandra	Kent	4134936183242102	9986.49	43548.71	436.08
100	Kevin	Taylor	4410369711798089	17029.56	66887.36	392.77

Result 80

select * from cards;

-- VI.B. List each card type and month in 2025, showing the total amount spent,

-- number of unique cardholders, and average transaction size for active cards, sorted

-- by card type and month.

-- ● Hint: Join transactions, accounts, and cards tables, filter for card_status and

-- year 2025, group by card_type and month from transaction_date, then

-- aggregate.

select c.card_type,month(str_to_date(transaction_date,'%Y-%m-%d')) as
months,sum(t.amount),count(distinct a.customer_id) as unique_cardholders,

round(avg(t.amount), 2) as avg_transaction_size

from cards as c

inner join accounts as a

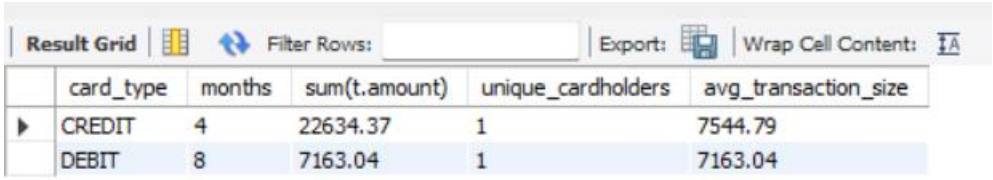
on c.account_id=a.account_id

inner join transactions as t

on a.account_id=t.account_id

where card_status='active' and year(str_to_date(transaction_date,'%Y-%m-%d'))=2025

group by c.card_type,month(str_to_date(transaction_date,'%Y-%m-%d'));



card_type	months	sum(t.amount)	unique_cardholders	avg_transaction_size
CREDIT	4	22634.37	1	7544.79
DEBIT	8	7163.04	1	7163.04

-- VI.C. List each card type and transaction type, showing the number of transactions,

-- total transaction amount, and average transaction amount, sorted by card type and

-- transaction count from highest to lowest.

-- ● Hint: Join cards, accounts, and transactions tables, group by card_type and

-- transaction_type, then compute the amount for each combination

select c.card_type,t.transaction_type,count(t.transaction_id) as no_of_transaction,sum(t.amount) as total_transaction_amount,avg(t.amount) as avg_transaction_amount

from cards as c

inner join transactions as t

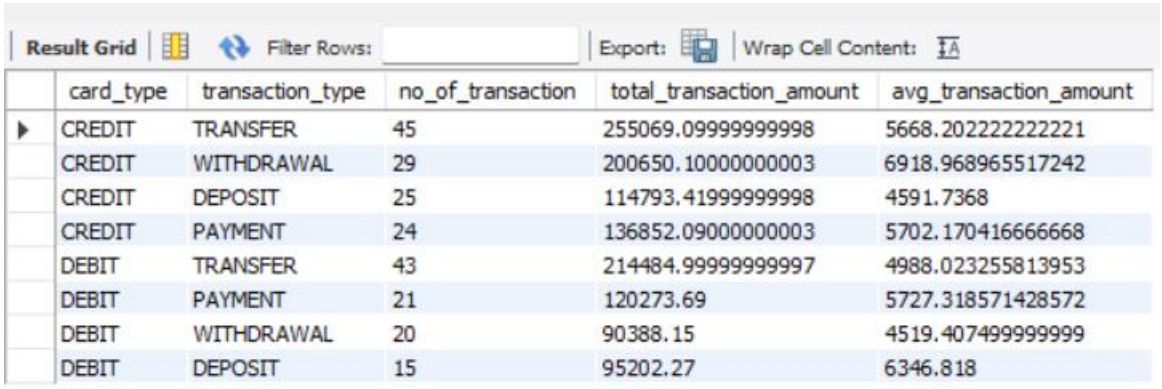
on c.account_id=t.account_id

inner join accounts as a

on a.account_id=t.account_id

group by c.card_type,t.transaction_type

order by c.card_type,no_of_transaction desc;



card_type	transaction_type	no_of_transaction	total_transaction_amount	avg_transaction_amount
CREDIT	TRANSFER	45	255069.09999999998	5668.202222222221
CREDIT	WITHDRAWAL	29	200650.10000000003	6918.968965517242
CREDIT	DEPOSIT	25	114793.41999999998	4591.7368
CREDIT	PAYMENT	24	136852.09000000003	5702.170416666668
DEBIT	TRANSFER	43	214484.99999999997	4988.023255813953
DEBIT	PAYMENT	21	120273.69	5727.318571428572
DEBIT	WITHDRAWAL	20	90388.15	4519.407499999999
DEBIT	DEPOSIT	15	95202.27	6346.818

-- VII. Actionable Insights & Recommendations

-- ● Hint:

-- ○ To formulate overall recommendations, start by synthesizing key

-- insights from each analysis section—customer behaviour, branch

-- performance, financial risk, and service interactions—to identify

-- patterns and opportunities.

-- ○ Focus on actionable strategies that address low engagement (e.g.,

-- incentives), high risk (e.g., default mitigation), and strong performance

-- (e.g., best-practice replication), tailoring them to specific customer

-- segments, branches, or timeframes.

-- ○ Ensure recommendations are practical, data-driven, and aligned with

-- business goals like retention, revenue growth, and risk management

1. Customer Engagement:

- The bank can offer cashback, discounts, and reward points to customers who make fewer transactions.
- This may encourage them to use their accounts and cards more frequently and improve customer retention.

2. Branch Performance:

- I would compare the performance of different branches to understand why some branches perform better than others.
- The bank can apply those successful practices to branches with lower transaction volumes or more customer complaints.

3. Loan Risk Management:

- The bank should regularly monitor overdue loan payments and identify customers who may be at risk of default.
- Early follow-ups and better credit assessment can help reduce loan defaults and financial losses.

4. Customer Service:

- I would focus on identifying the most common customer complaints and the reasons behind them.
- Based on these findings, the bank can improve employee training and reduce complaint resolution time.

5. Card Usage:

- The bank can analyze the usage patterns of different card types to understand which cards are used more frequently.
- Based on customer preferences, it can promote relevant card benefits and encourage more transactions.

6. Performance Monitoring:

- The bank should regularly track important metrics such as transaction volume, revenue, loan defaults, and complaint rates.
- Comparing these metrics over time will help measure improvements and identify areas that need further attention.

Conclusion:

Overall, my analysis helped me understand how customer behaviour, branch performance, loan risk, and customer service are connected to the bank's performance. By focusing on these areas and making data-driven decisions, the bank can improve customer retention, increase revenue, provide better service, and manage financial risks more effectively.